

MLOps

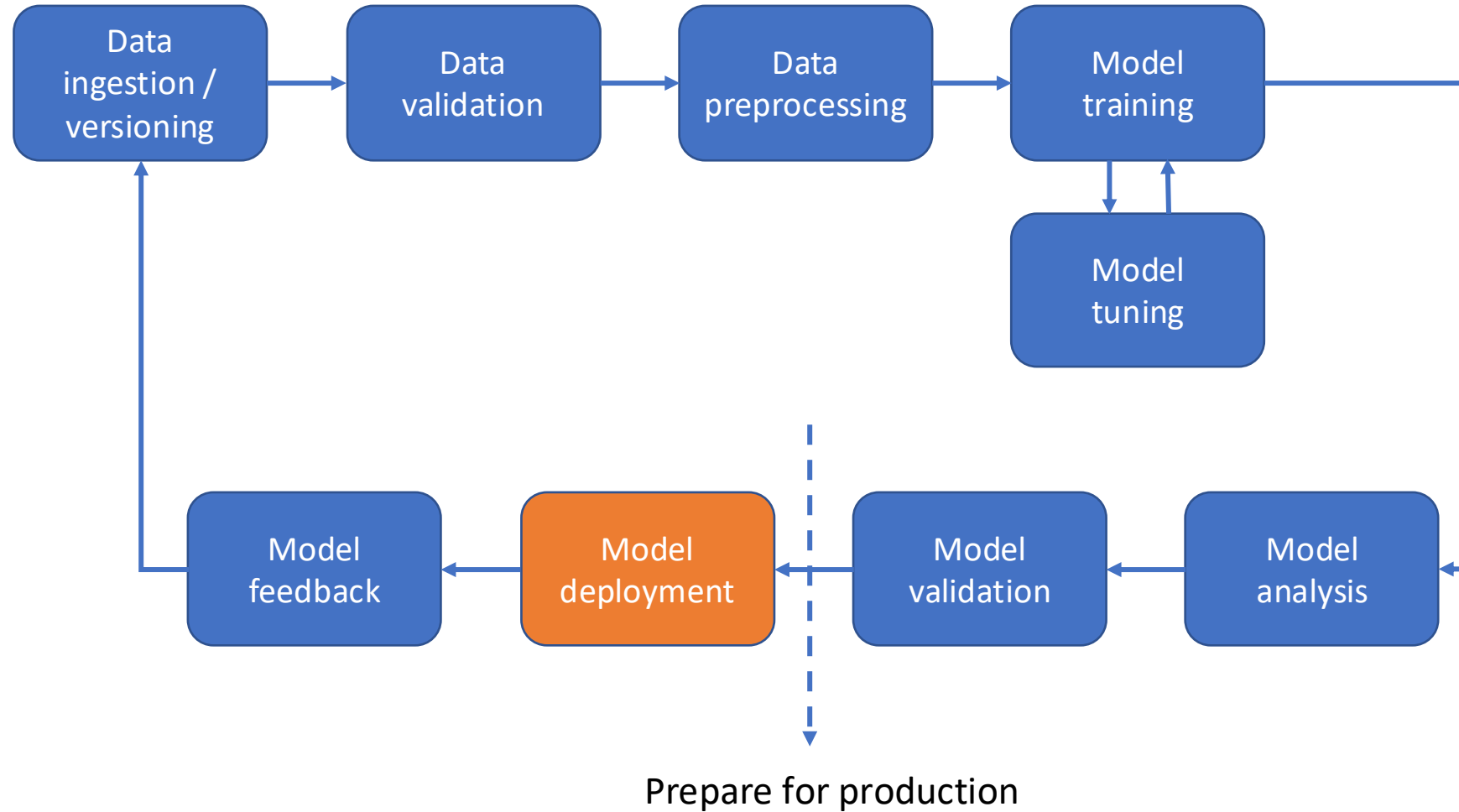
Serverless Deployment – Monitoring

Session 7

Juan Ramírez
jramirezg@faculty.ie.edu

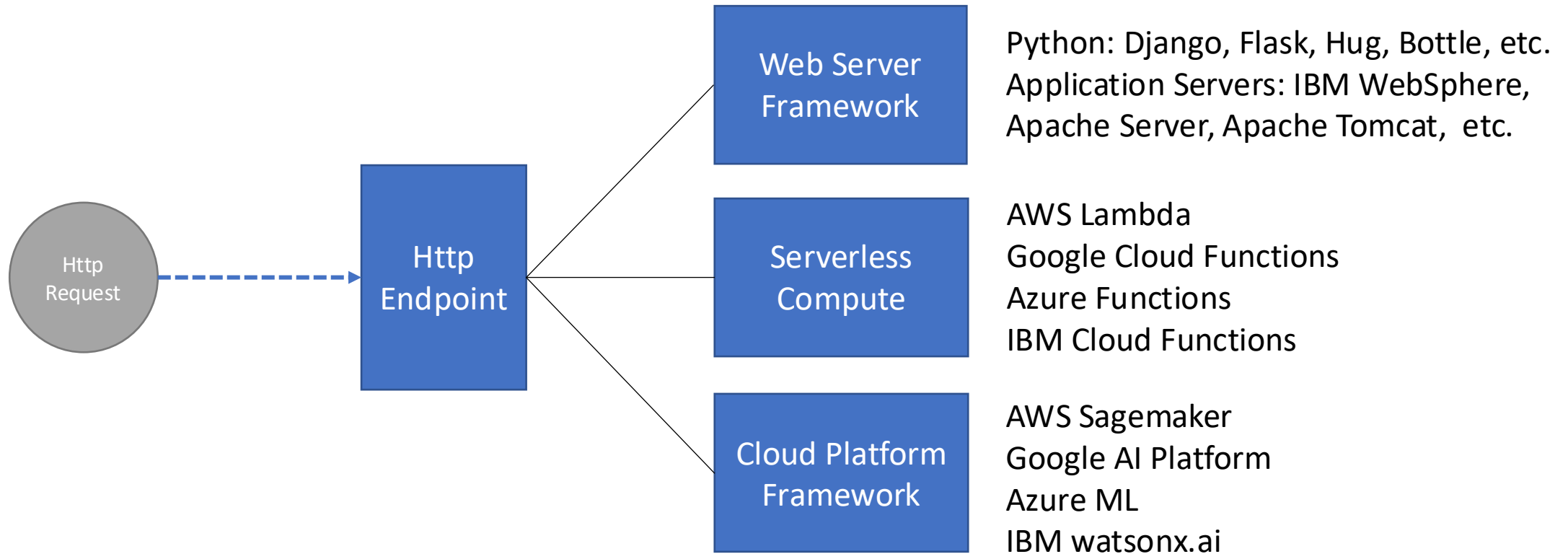
Serverless Deployment

Model Deployment



Serverless Deployment Models for prediction

- We can deploy our ML model in 3 ways:



What is Serverless Deployment?



- Next step in Cloud Computing
- The separation between server and application is managed by using a platform
- The responsibility of the platform or serverless provider is to manage all the needs and configurations for your application.
- These platforms manage the configuration of your server behind the scenes.
- This is how in serverless computing, one can simply focus on the application or code itself being built or deployed
- The service will take care of running it, and it will scale it up and down according to the load.
- Additionally, you need to pay only for the time when the function is actually used.

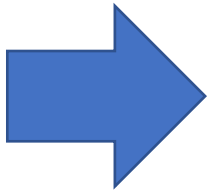
Serverless Architecture Overview

Function as a Service (FaaS)

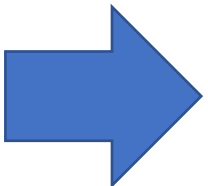
- Functions solving specific tasks
- Functions can be triggered from events (HTTP requests, changes in serverless databases, etc.)
- Functions deactivate once their task is complete.

Backend as a Service (BaaS)

- Developers delegate their backend processes to a third-party vendor
- Remote servers implement and store the app's backend.



Developers concentrate writing code without worrying about infrastructure and server management.



Significant increase in serverless adoption in recent years (nearly 40% companies worldwide using it in some form)

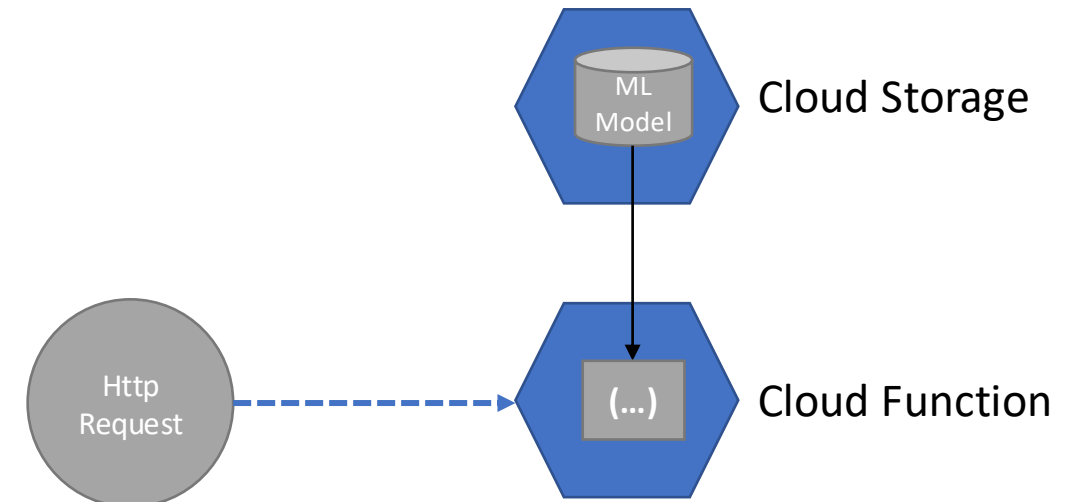
How Serverless Deployment Work



- **Code deployment:** Developers upload their code repository to a serverless deployment platform. This code defines specific functions and their triggers – anything from a user clicking a button to an HTTP request.
- **Event triggering:** An event like a user's click triggers the specific function's activation.
- **Resource allocation:** The serverless platform automatically provides the computing resources necessary for executing the function.
- **Function execution:** At this point, the function runs and completes its designated task.
- **Resource release:** After completing the task, the platform releases resources. Then, the function deactivates.

Steps

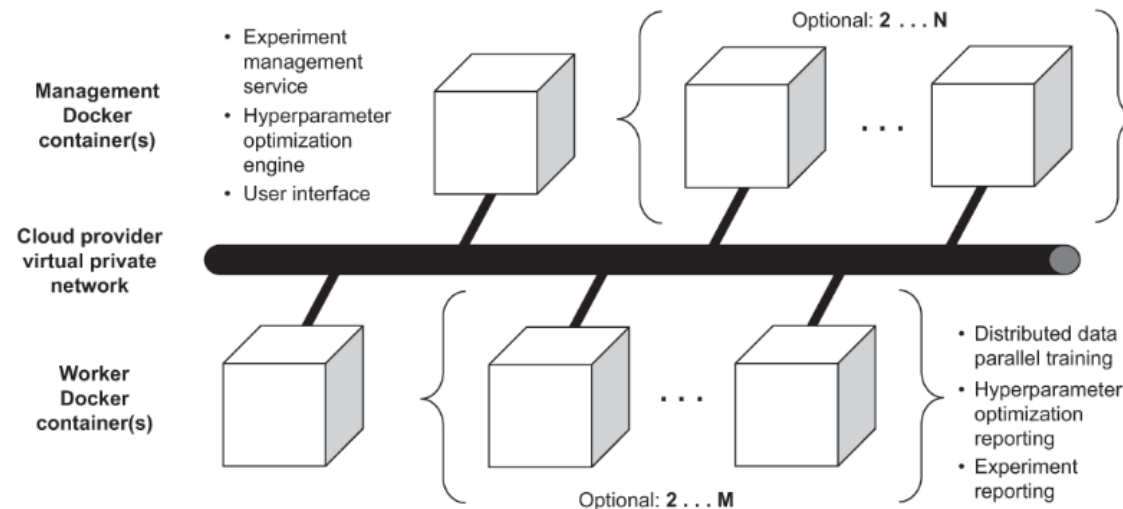
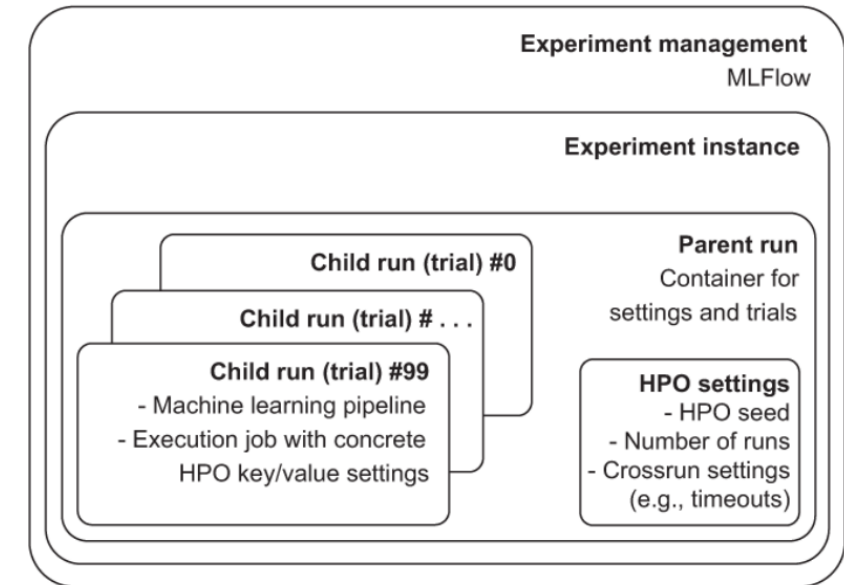
1. Save model to bucket in cloud storage
2. Build Cloud Function
3. Deploy the model exposing endpoint



An example of distributed ML Pipeline

Frameworks used

- **MLFlow**—For experiment management
- **Optuna**—For hyperparameter optimization
- **Docker**—For pipeline component packaging and reproducible execution
- **PyTorch Lighting**—For PyTorch machine learning model training and validation
- **Kaen**—For provisioning and management of the pipeline across public cloud providers



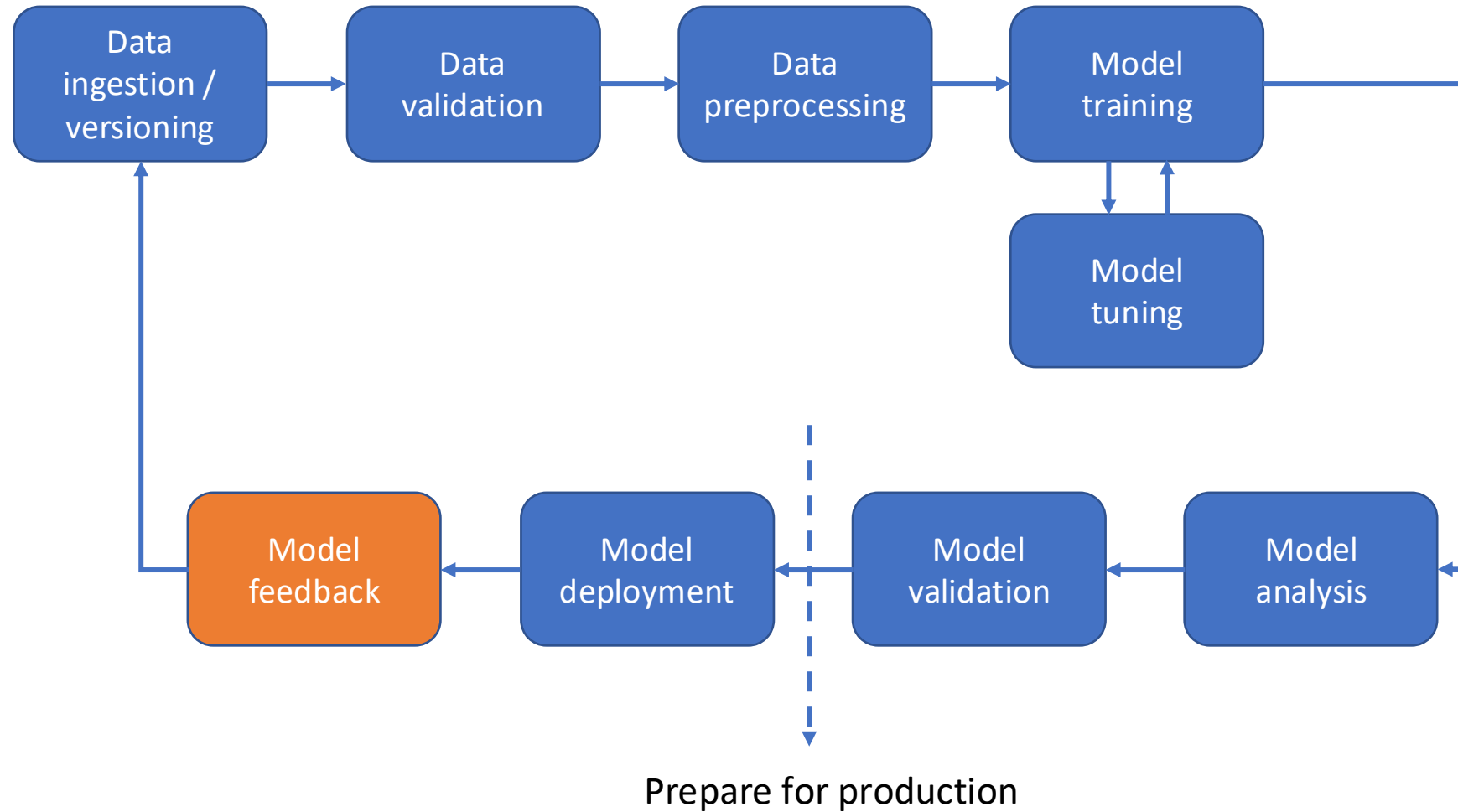
Provisioning of the nodes, machine learning middleware (experiment management, hyperparameter optimization, and so on), as well as the orchestration of the machine learning pipeline execution across the worker nodes, are handled entirely by the Kaen framework and associated Docker containers

Source: MLOps Engineering at Scale (Carl Osipov, 2022)

AWS Lambda – IBM Watson Machine Learning

Model Monitoring

Model Feedback



Monitoring and feedback loop



Machine learning models need to be monitored at three levels:

IT

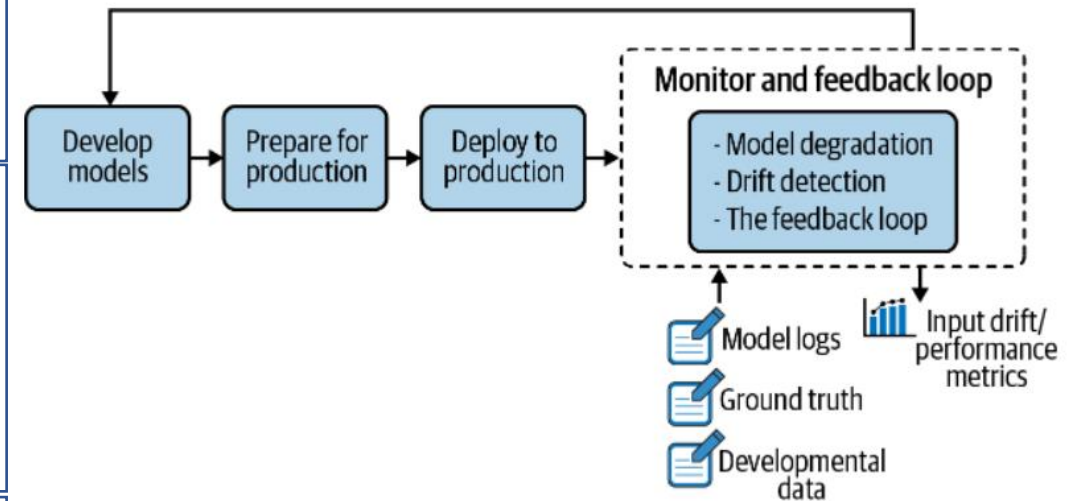
- **At the resource level**, including ensuring the model is running correctly in the production environment.
 - Key questions include:
 - Is the system alive?
 - Are the CPU, RAM, network usage, and disk space as expected?
 - Are requests being processed at the expected rate?

Data
Science

- **At the performance level**, meaning monitoring the pertinence of the model over time.
 - Key questions include:
 - Is the model still an accurate representation of the pattern of new incoming data?
 - Is it performing as well as it did during the design phase?

KPIs

- **At the business level**, meaning monitoring the relevance of the business goals
 - Key questions:
 - Is the model delivering value to the enterprise?
 - Is the model helping to achieve our goals?



Source: Introducing MLOps: How to Scale Machine Learning in the Enterprise (Mark Treveil & Dataiku Team, O'Reilly Media, Inc., 2020)

How often should models be retrained?

- No easy answer
- There are several factors that may impact:
 - The domain
 - Is it a rapid changing domain? -> cybersecurity, real-time trading
 - The cost
 - If it takes one week to run the whole data pipeline and retrain the model, is it worth a 1% improvement?
 - The training data
 - Some situations are constrained by the limited number of training examples

Some level of model retraining is **definitely necessary**: it's not a question of if, but of when

Ground Truth vs. Input Drift

Ground Truth

- The ground truth is the correct answer to the question that the model was asked to solve
- Sometimes ground truth is obtained rapidly after a prediction—for example, in models that decide which advertisements to display to a user on a web page. The user is likely to click on the advertisements within seconds, or not at all.
- However, in many use cases, obtaining the ground truth is much slower. If a model predicts that a transaction is fraudulent, how can this be confirmed?



Input Drift

- Input drift is based on the principle that a model is only going to predict accurately if the data it was trained on is an accurate reflection of the real world.
- If a comparison of recent requests to a deployed model against the training data shows distinct differences, then there is a strong likelihood that the model performance is compromised.
- The beauty of this approach is that all the data required for this test already exists, so there is no need to wait for ground truth or any other information.

Ground Truth Evaluation

- Ground truth retraining requires **waiting for the label event**.
 - For example, in a fraud detection model, the ground truth would be whether or not a specific transaction was actually fraudulent.
 - For a recommendation engine, it would be whether or not the customer clicked on—or ultimately bought—one of the recommended products.
- With the new ground truth collected, the next step is to compute the performance of the model based on ground truth and compare it with registered metrics in the training phase.
- When the difference surpasses a **threshold**, the model can be deemed as outdated, and it should be retrained.
- The metrics to be monitored can be of two varieties:

Statistical metrics

- Accuracy, ROC AUC, log loss, etc.
- First choice is the metric chosen when the model was created
- For more complex models, where the average performance is not enough, it may be necessary to look at metrics computed by subpopulations.

Business metrics

- Cost-benefit assessment
- For example, the credit scoring business has developed its own specific metrics.

Large Language Models Monitoring: Use Case Oriented

• Text Summarization Metrics

- ROUGE
- SARI
- Normalized F1, Precision, Recall
- METEOR
- Sentence Similarity - Jaccard Similarity, Cosine Similarity
- BLEU
- Readability, complexity
- HAP Detection on Input and Output Text
- PII Detection on Input and Output Text

Q&A Evaluation Metrics

- ROUGE
- BLEU
- METEOR
- exact_match
- HAP Detection on Input and Output Text
- PII Detection on Input and Output Text

Entity Extraction Metrics

- Micro & Macro F1, Precision, Recall
- Rouge
- Exact Match

• Quality/Text Classification Metrics

- Accuracy
- Precision
- Recall
- ROC AUC
- F1 Score
- Matthews Correlation Coefficient
- Label Skew

Content Generation Metrics

- ROUGE
- BLEU
- METEOR
- exact_match
- Readability, complexity
- HAP Detection on Input and Output Text
- PII Detection on Input and Output Text

Drift Monitoring (applicable for all Task Types)

- Metadata Drift (applicable for both Input and Output content)
- Context Drift
- Confidence Drift
- Distribution Drift

Explainability Monitoring

- Attributions Detection using RAG

Model Health Metrics (applicable for all Task Types)

- Total Scoring Requests
- Number of scoring records
- Input token count
- Output token count
- API Latency
- API Throughput
- Record Latency
- Record Throughput
- User count

Challenges with Ground Truth

Ground truth is not always immediately, or even imminently, available

- For some types of models, teams need to wait months (or longer) for ground truth labels to be available, which can mean significant economic loss if the model is degrading quickly
- Deploying a model for which the drift is faster than the lag is risky
- However, by definition, drifts are not forecastable, so models with long lags need mitigation measures.

Ground truth and prediction are decoupled

- To compute the performance of the deployed model on new data, it's necessary to be able to match ground truth with the corresponding observation
- In many production environments, this is a challenging task because these two pieces of information are generated and stored in different systems and at different timestamps
- For low-cost or short-lived models, it might not be worth automated ground truth collection -> **Risky**

Ground truth is only partially available

- In some situations, it is extremely expensive to retrieve the ground truth for all the observations, which means choosing which samples to label and thus inadvertently introducing bias into the system

Causes of Data Drift



Sample selection bias, where the training sample is not representative of the population

- For instance, building a model to assess the effectiveness of a discount program will be biased if the best discounts are proposed for the best clients.
- Selection bias often stems from the data collection pipeline itself.
- Sometimes it is not easy to collect non-biased data as the problematic features are often unknown or maybe even not available.

Non-stationary environment, where training data collected from the source population does not represent the target population

- This often happens for time-dependent tasks—such as forecasting use cases—with strong seasonality effects, where learning a model over a given month won't generalize to another month.

Input Drift Detection Techniques

Univariate statistical tests

Statistical test on data from the source distribution and the target distribution for each feature.

Two tests:

- For continuous features, the **Kolmogorov-Smirnov test** is a nonparametric hypothesis test that is used to check whether two samples come from the same distribution. It measures a distance between the empirical distribution functions.
- For categorical features, the **Chi-squared test** is a practical choice that checks whether the observed frequencies for a categorical feature in the target data match the expected frequencies seen from the source data.

The main advantage of p-values is that they help detect drift as quickly as possible.

The main drawback is that they detect an effect, but they do not quantify the level of the effect (i.e., on large datasets, they detect very small changes, which may be completely without impact).

Input Drift Detection Techniques

Domain classifier

- Train a model that tries to discriminate between the original dataset (input features and, optionally, predicted target) and the production dataset, i.e. a model that aims at predicting the data's origin.
- The performance of the model (its accuracy, for example) can then be considered as a metric for the drift level.
- If this model is successful in its task, and thus has a high drift score, it implies that the data used at training time and the new data can be distinguished, so it's fair to say that the new data has drifted.
- To gain more insights, in particular to identify the features that are responsible for the drift, one can use the feature importance of the trained model.

The Feedback Loop

- Feedback loop: information from the production environment flows back to the model prototyping environment for further improvement.
- Three main components



Logging system

collects data from several production servers

Model evaluation store

versioning and evaluation between different model versions

Model comparison on production environments

either with the shadow scoring (champion/challenger) setup or with A/B testing

Evidently – IBM Watson OpenScale